



Software Engineering Foundations

ENTERPRISE MOBILE APPLICATION DEVELOPMENT BOOTCAMP

PHASE 01

Software Engineering Foundations for Mobile

Phase 01 - Three Modules at a Glance

Module	Schedule	Core Skills
Module 1 Software Engineering Principles	Day 1 & 2	SDLC · Agile/Scrum · Platform strategy · SOLID (all 5) · Clean code · Technical debt · User stories
Module 2 Source Control and Team Development	Day 3 & 4	Git internals · Core operations · GitFlow / TBD / GitHub Flow · Conventional Commits · PRs · Code review · Conflicts · Advanced Git
Module 3 Problem Solving and Engineering Thinking	Day 5	Decomposition · Technical communication · Estimation · 7-step debugging · Root cause analysis · Architecture whiteboarding



Modern Software Engineering Principles

Module 1

The Software Development Lifecycle

THE PROBLEM

Why can't we just start coding?

Every project that skips planning pays for it in rework, missed requirements, and production defects.

The Seven SDLC Phases

Phase	What Happens	What Gets Skipped = What Goes Wrong
Planning	Scope, resources, risks, feasibility	Scope creep, missed deadlines, wrong team size
Requirements	What the software must do (functional) and how (NFRs)	Building the wrong thing. Rework.
Design	Architecture, API contracts, data models, UI flows	Incompatible components. Integration surprises.
Development	Code, review, integrate	Without prior phases: chaos
Testing	Correctness, performance, security, usability	Production defects. Loss of user trust.
Deployment	Release through a controlled process, CI/CD	Unreproducible deployments. Rollback failures.
Maintenance	Monitor, fix, iterate - 60–80% of lifecycle cost	Technical debt accumulation. Slow teams.



SDLC Models - Which Fits Your Project?

- Waterfall: sequential, each phase completes before the next - rigid, but right for fixed regulatory specs
- Iterative: phases repeat in cycles - earlier feedback, some flexibility
- Agile / Scrum: two-week sprints, working software every cycle - dominant model for enterprise mobile
- Spiral: risk-driven iterations - used in safety-critical systems (medical, aerospace)
 - Is a meta-model, combining features of Waterfall, Agile, and Iterative
- This program uses Agile/Scrum throughout - the model you will use in every Phase lab



Which SDLC Phase Gets Skipped Most?

Skill Check ✓

Your Turn

Which phase do you think teams skip most often in real projects? What happens as a result?



Agile and the Scrum Framework

THE AGILE MANIFESTO VALUES (2001)

Individuals and interactions over processes and tools.

Working software over comprehensive documentation.

Customer collaboration over contract negotiation.

Responding to change over following a plan.

The items on the right have value - we just value the items on the left more.

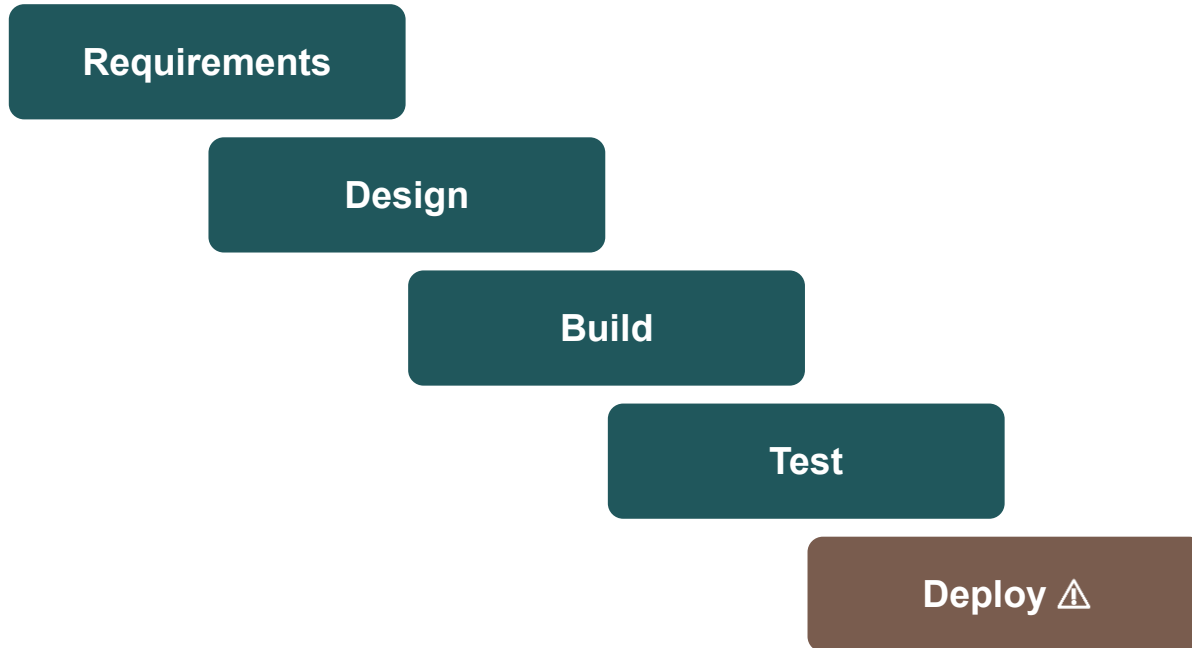
The 12 Agile Principles - Operational Meaning

#	Principle	What It Means in Mobile Engineering
1	Early and continuous delivery of working software	Builds to TestFlight/Firebase every Sprint - not demos, not prototypes
9	Technical excellence and good design enhance agility	SOLID, clean code, tests - not opposed to speed, they enable it
10	Simplicity - maximizing the work NOT done	Build what's needed now. Over-engineering is as dangerous as under-engineering
12	Reflect and adjust at regular intervals	The retrospective operationalizes this - it's not optional



Waterfall Cascades. Scrum Loops.

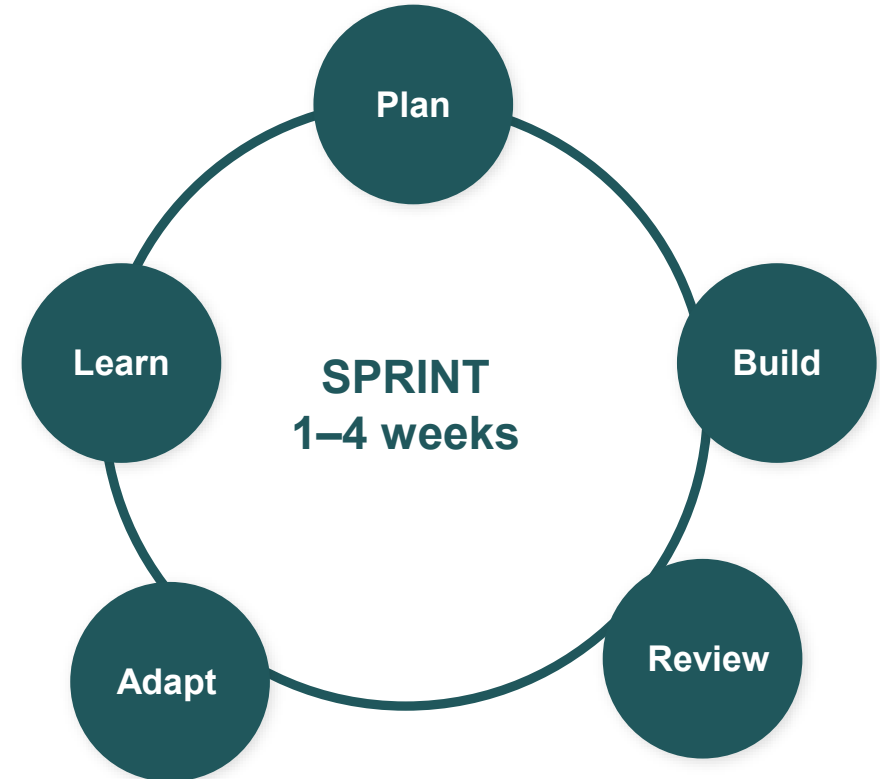
WATERFALL - value arrives at the end



⚠ Feedback arrives here - months later

Change is expensive. Risk is discovered late. Stakeholders see the product once.

SCRUM - value arrives every Sprint



✓ Working product + feedback every cycle

Change is cheap. Risk surfaces early. Stakeholders see real results continuously.



Scrum Roles - What They Are and Are Not

Role	Core Responsibility	What They Are NOT
Product Owner	Owns and orders the backlog. Single point of contact for business requirements.	Not a requirements gatherer. Not someone who can be unavailable for days.
Scrum Master	Serves the team. Removes impediments. Protects the process. Coaches.	Not a project manager. Not a task-assigner. Not a secretary.
Developers	Create the increment. Collectively accountable for quality and delivery.	Not just coders - everyone who contributes to the increment.



Three Accountabilities. One Team. Zero Hierarchy.



Product Owner

Owns VALUE

- Orders the Product Backlog
- Makes prioritization decisions
- Carries the product vision
- One person - not a committee



Scrum Master

Owns EFFECTIVENESS

- Coaches the team on Scrum
- Removes impediments
- Shields team from interference
- Leads by serving - not directing



Developers

Own the HOW

- Everyone building the Increment
- Self-managing - no task assignment
- Collectively own quality (DoD)
- Cross-functional by design



The Product Owner: Value Decision-Maker



The PO is accountable for...

- Maximizing the value of the product
- Owning and ordering the Product Backlog
- Deciding what to build next - by value and risk
- Communicating the product vision and goal
- Being available to the team for fast answers



The PO is not...

- A requirements writer who hands off a spec
- A committee or a rotating seat
- A task assigner directing the Developers
- A proxy who must escalate every decision
- Optional - an absent PO starves the team



The Scrum Master: Coaching, Not Policing



Serves the team

Removes impediments blocking flow; creates the conditions for focus



Serves the Product Owner

Helps with backlog techniques and stakeholder collaboration



Serves the organization

Coaches leadership on how to work with - not over - Scrum teams

Never: assigns work · manages people · reports team status upward

When the SM starts doing these, self-management dies - and Scrum with it.



“Who does status reporting, then?”

Nobody - because the artifacts and events ARE the status. The Sprint Review shows working product. The backlog shows what's done and what's next. Transparency by default, not by report.



Developers: Everyone Who Builds the Increment

“Developer” is not a job title in Scrum - it's anyone whose work creates the Increment: engineers, testers, designers, analysts, writers, data scientists.



Cross-functional accountability

The team collectively owns delivery - no “not my job” inside the Sprint



They create the Sprint plan

The PO says what and why. The Developers decide how - and estimate it



Quality lives with the team

The Definition of Done is enforced by the people doing the work



Self-management, in reality

Not the absence of management - management that lives inside the team rather than above it



Three Artifacts, Three Commitments



WHAT + WHY

Product Backlog

The single, ordered list of everything that might improve the product. Never complete - it evolves as the team learns.

Commitment: Product Goal



THIS SPRINT

Sprint Backlog

The Sprint Goal, the selected items, and the team's plan. Owned by the Developers and updated daily.

Commitment: Sprint Goal



WHAT'S REAL

Increment

All completed work - usable, meeting the quality bar. If it doesn't meet the DoD, it isn't done.

Commitment: Definition of Done



The Product Backlog: What Belongs, What Doesn't

Belongs in the Product Backlog

- ✓ Features and user stories
- ✓ Defects and technical debt
- ✓ Compliance and regulatory items
- ✓ Research spikes and experiments
- ✓ Improvements surfaced by Retros

Doesn't belong

- ✗ Task-level breakdowns (that's the Sprint Backlog)
- ✗ Items no one intends to ever do
- ✗ Duplicate wish lists from every stakeholder
- ✗ Hidden side-channel work



Order for value and risk - not for politics.

High value + high risk items rise to the top: learn the hard things early, while change is still cheap.



Sprint Backlog



Sprint Backlog

A forecast, not a contract

“We believe we can do this” - updated daily as the team learns

Owned entirely by the team

No outside party adds, removes, or verifies items

Transparency, not control

Visible to everyone; managed by no one above the team



Increment - Delivered Working Product



Definition of Done

“Done” is the most dangerous word in delivery. Coded but not tested? Tested but not deployed? Deployed but not accepted?

The DoD is a quality contract:

- One shared standard for “complete”
- Applied to every item, every Sprint
- If it doesn't meet the DoD - it is not done. Period.



Writing User Stories and Acceptance Criteria

The INVEST Criteria - What Makes a Good Story

Letter	Characteristic	What it Means
I	Independent	The story can be developed without depending on other stories
N	Negotiable	The story is a placeholder for conversation, not a detailed contract
V	Valuable	The story delivers value to a user, customer, or stakeholder
E	Estimable	The team understands it well enough to estimate the effort
S	Small	The story is small enough to fit comfortably within a sprint
T	Testable	The team can verify whether the story is done



From BRD Statement to User Story

BRD STATEMENT

"The system shall allow users to filter search results by date range, category, and price."



USER STORY

As a **customer**, I want to **filter my search results by date range, category, and price**, so that **I can select the right product for my needs**.

The anatomy - and why each part matters

WHO

As a [type of user]

Keeps a real human in the frame - not "the system"

WHAT

I want to [action]

The capability, stated in user language

WHY

So that [benefit]

The value test - if you can't fill this in, ask why it's in the backlog



Writing Acceptance Criteria

- Acceptance criteria define the conditions under which a user story is considered complete
- They should be included in a user story that is ready for development
 - Definition of Ready
- The Gherkin syntax (Given/When/Then) structures them as testable scenarios:
 - Given: The preconditions or context (the world's state before the action)
 - When: The action the user takes or the event that occurs
 - Then: The expected outcome - the system's new state after the action
 - And: Additional conditions chained to any of the above

Why Gherkin?

Gherkin scenarios can be directly automated using Cucumber, XCTest (iOS), or Espresso (Android). A story with complete Given/When/Then scenarios is 80% of the way to an automated acceptance test suite. Writing them forces precision in requirements before any code is written.



Given / When / Then - Acceptance Criteria

Story: As a field technician, I want to log hours offline so that I can work in areas with no connectivity.

Scenario 1: Logging while offline

Given the device has no network connectivity and the user is on the Time Entry screen

When the user enters 4.5 hours for Project Alpha and taps Save

Then the entry is saved to local storage, and the entry appears in the list with a 'Pending Sync' label

Scenario 2: Sync on reconnect

Given the device has 3 entries with 'Pending Sync' status

When the device regains network connectivity

Then the app automatically syncs all pending entries and successfully synced entries update their status to 'Submitted'



Common User Story Anti-Patterns

Anti-Pattern	Example	Problem	Fix
Technical task masquerading as a story	"As a developer, I want to refactor the network layer so that it uses protocols."	No user value. The 'actor' is the developer, not a system user.	Developers are not story actors. Track technical work as tasks or enablers with clear rationale tied to user value.
Missing business value	"As a manager, I want to see a dashboard."	No 'so that' clause. Why do they want a dashboard? What decision does it enable?	"...so that I can identify which employees have not submitted timesheets before the Friday deadline."
Feature-as-story (too large)	"As a user, I want to manage my account."	Unestimable. Includes update email, change password, update profile photo, deactivate account, set notification preferences - each a separate story.	Split into individual capabilities, each independently valuable.
Prescribes implementation	"As a user, I want a modal dialog that appears when I tap the + button."	Specifies the solution, not the need. Design is not the story author's domain.	"As a user, I want to add a new time entry from the timesheet screen." (Let designers and engineers decide the interaction pattern.)



User Story Writing Workshop

Skill Check ✓

- Rewrite the following Business Requirements as User Stories, complete with acceptance criteria:
- Feature request: 'The app should let managers see who hasn't submitted their timesheet yet.'
- Feature request: 'We need the app to handle when field technicians work in areas with no cell service.'
- Feature request: 'Employees should be able to correct a timesheet after submission if they made a mistake.'



Scrum Ceremonies - Purpose and Anti-Patterns

Ceremony	Time-box	Purpose	Most Common Anti-Pattern
Sprint Planning	4 hrs (2-wk sprint)	Sprint Goal + Sprint Backlog	Planning without a Sprint Goal - just a list of tasks
Daily Scrum	15 min	Inspect progress toward Sprint Goal	Status report to manager instead of peer sync
Sprint Review	2 hrs (2-wk sprint)	Demo Done work to stakeholders, adapt backlog	Showing not-Done work. No stakeholders present.
Sprint Retrospective	1.5 hrs	Improve process - specific actions with owners	12 items with no owners and no due dates



Five Events, One Continuous Loop



Timeboxes (for a 1-month Sprint)

Sprint Planning

8 hrs max

Daily Scrum

15 min - always

Sprint Review

4 hrs max

Retrospective

3 hrs max

Shorter Sprints scale these down proportionally. The timebox is a ceiling, not a target.



Sprint Planning

Sprint Planning answers three questions

WHY is this Sprint valuable?

The PO proposes; the team crafts the Sprint Goal together

WHAT can be done this Sprint?

Developers pull items from the top of the backlog - they choose the amount

HOW will the work get done?

The team plans its own approach. No one plans it for them



The Daily Scrum

The Daily Scrum is NOT a status meeting

15 minutes, daily, for the Developers

Purpose: inspect progress toward the Sprint Goal and adapt the plan

No one reports to anyone - the team coordinates with itself

SM and PO attendance: optional

The 3-question script is a guideline, not a rule - many teams walk the board instead

Each team member answers three questions:

WHAT have you done since yesterday?

WHAT are you planning for today?

WHAT roadblocks are you currently facing?



Sprint Review



Sprint Review

Inspects the **PRODUCT**

- Working Increment shown to real stakeholders
- Feedback flows into the Product Backlog
- Not a polished demo - an honest inspection
- Replaces the “big reveal” at the end of a Waterfall project

Compare: in Waterfall, stakeholders often see the product for the first time at UAT - when change is most expensive.



Sprint Retrospective



Sprint Retrospective

Inspects the **PROCESS**

- How did we work together this Sprint?
- What will we improve - concretely - next Sprint?
- The engine of continuous improvement
- Most skipped when pressure is high - exactly when it's most needed

A Retro that produces the same action items every Sprint isn't a Retro. It's a ritual.



Effort Estimation and Planning Poker

Scrum Does Not Require Story Points

The Scrum Guide
mentions story points...

0 times

Scrum requires that the team can size work well enough to make a credible Sprint forecast. How is entirely up to the team.

When estimation helps

- ✓ Forces conversation that surfaces hidden complexity
- ✓ Builds shared understanding before work starts
- ✓ Supports credible forecasting for stakeholders

When estimation hurts

- ✗ Debating points consumes more time than the work
- ✗ Estimates get treated as commitments - then weaponized
- ✗ Precision theater: false confidence in unknowable numbers



Common Estimation Techniques

Story Points

Relative sizing (Fibonacci). Measures complexity, not hours.

Best for: Teams wanting effort decoupled from time

Planning Poker

Simultaneous reveal prevents anchoring. Discussion on splits.

Best for: Surfacing disagreement that hides complexity

T-Shirt Sizing

S / M / L / XL. Fast, low-argument, roughly right.

Best for: Early grooming and roadmap-level planning

Right-Sizing

Split everything until it's "a day or two." Count items.

Best for: Mature teams who want radical simplicity

Raw Time

Hours or days. Familiar and simple.

Best for: Teams new to Scrum or non-software contexts

Throughput

Forecast from historical items-per-Sprint data.

Best for: Stable teams with consistent item sizes



Story Points - What They Mean in Practice

Points	Meaning	Mobile Example
1	Trivial. Completely understood. No unknowns.	Change an error message. Fix a typo. Update a button color.
2	Small, well-understood. One component.	Add a loading spinner to an existing screen. Show one additional field.
3	Moderate. 2–3 components. Straightforward tests.	New form with validation and API call. Filter option on existing list.
5	Significant. Multiple components. Integration testing needed.	New screen from one API endpoint - loading, error, empty, populated states.
8	Complex. Cross-cutting. Significant unknowns.	Push notification handling with deep link. Offline-capable screen.
13	Very complex. Architecture implications. Multiple unknowns.	Offline sync with conflict resolution. Biometric auth on existing flow.
21	Too large. Must be decomposed before committing.	Any 21-point story should be split. If it won't split, spike first.



Planning Poker - Mechanics and Value

- PO reads the story. Team asks clarifying questions. Questions stop when no new information is coming.
- Each team member privately selects their estimate card.
- All cards revealed **SIMULTANEOUSLY** - this prevents anchoring (the first number heard biases everyone else).
- If estimates are identical or within one Fibonacci value: adopt. Brief discussion.
- If estimates diverge significantly: HIGH estimator explains first, then LOW. Discussion follows.
- Revote after discussion if needed. Typically 1–2 rounds to consensus.
- The most valuable moment: when a 3 and a 13 appear on the table. Don't rush to consensus - the divergence is the information.



7 Estimation Anchors - Questions to Ask Before Voting

Question	Direction if YES
Have we built something similar before?	YES → lower estimate. NO → higher estimate or spike.
Does this require a spike first?	YES → do not estimate this story yet. Spike it first.
Does this touch iOS AND Android?	YES → multiply, don't just add. Platform parity is real cost.
Does this touch multiple services or teams?	YES → add coordination overhead - meetings, waiting, alignment.
Are there security or compliance requirements?	YES → add significant weight. Security review is never free.
Are there performance requirements needing profiling?	YES → add time for measurement and tuning cycles.
Are there accessibility requirements?	YES → add implementation and VoiceOver/TalkBack test time.



Story Point Estimation - Four Stories

Story 1

Add a note field to an existing time entry form.

Story 2

Daily summary push notification for managers.

Story 3

GPS geofencing to auto-detect shift start/end.

Story 4

Configurable overtime threshold per contract type.



Plan your Sprint from this Backlog

Skill Check ✓

#	User Story	Points	Priority	In Sprint 1?
1	As a user, I can log in with SSO credentials	?	Critical	Y / N
2	As a user, I can view my account dashboard	?	High	Y / N
3	As a user, I can update my profile information	?	High	Y / N
4	As an admin, I can assign user roles	?	Medium	Y / N
5	As a user, I can reset my password via email	?	High	Y / N
6	As a user, I can view my transaction history	?	Medium	Y / N
7	As a user, I receive email notifications on order status	?	Medium	Y / N
8	As an admin, I can export user data to CSV	?	Low	Y / N
9	As a user, I can set notification preferences	?	Low	Y / N
10	As a user, I can access help documentation	?	Low	Y / N

